

StayFree — Smart Notes Feature Implementation Roadmap

Document Overview

This document outlines the complete phased implementation plan for adding Smart Notes and Command Mode features to StayFree, transforming it from a dictation-only app into an AI-powered personal assistant with intelligent note-taking capabilities.

Current State: StayFree is a macOS voice dictation tray app (hold Left Option → record → transcribe → format → auto-paste)

Target State: Wispr Flow-level dictation + [Mem.ai/Voicenotes-level](#) smart notes in a single tray app

Total Implementation Phases: 4

Phase 1 — Smart Notes Foundation (Regex-based, Minimal AI)

Goal

Turn StayFree from "dictation-only" into "dictation + solid notes hub" with zero UI friction.

Core Components

1. Data Model

- **Notes storage** in electron-store
- **Note interface** structure:
 - id: crypto.randomUUID()
 - title: Auto-generated from first ~60 chars, user-editable
 - content: Formatted note text
 - rawContent: Original raw transcription

- createdAt: Date.now()
- updatedAt: Date.now()
- source: "voice" | "text" | "clipboard" | "transcription"
- pinned: boolean
- archived: boolean
- tags: string[] (empty initially, used in Phase 2)
- **Storage limit:** 500 notes maximum

2. Command Mode Hotkey Setup

- **Right Option key** detection (separate from Left Option dictation)
- **Visual distinction** in widget:
 - Dictation mode (Left Option): Red color
 - Command mode (Right Option): Purple/blue color
- **Recording pipeline:** Same audio capture, different post-processing path

3. Voice Commands (Regex-based Parsing)

Command patterns:

Voice Command	Action
"save to notes: [content]"	Create note with spoken content
"note: [content]"	Create note with spoken content
"save clipboard to notes"	Read clipboard → create note
"open notes" / "show notes"	Open Settings window on Notes tab

Table 1: Phase 1 Command Mode voice commands

Implementation approach: Fast regex pattern matching (0ms vs 200-400ms LLM parsing), predictable behavior.

4. Notes Page UI

Layout structure:

- Top section: Text input field + "+" button for keyboard note creation
- Pinned section: Horizontally scrollable cards for pinned notes
- Recents section: Main notes list with search bar + view toggle
- Empty state: Helpful message teaching Command Mode usage

Note Card components:

- Title (auto-generated, editable)
- 2-line content preview
- Timestamp
- Source icon (microphone/keyboard/clipboard/history)
- Hover actions: pin, archive, delete, copy

Note Detail Modal:

- Editable title (click to edit)
- Editable content (textarea)
- Tags display (manual in Phase 1)
- Metadata: created date, source type
- Actions: copy, delete, archive

Search functionality:

- Debounced search bar (300ms delay)
- Case-insensitive keyword matching
- Searches across: title, content, tags

View options:

- Grid view (cards)
- List view (compact rows)

5. Transcription History Integration

"Save as Note" feature:

- Bookmark icon button in transcription history rows
- Click → converts history entry to note
- Source tagged as "transcription"
- Preserves original raw content

6. Notes CRUD Service

Core functions (src/main/notes.ts):

- `createNote(params)` → Generate UUID, auto-title, prepend to array, cap at 500

- `updateNote(id, updates)` → Partial update (title, content, pinned, archived, tags)
- `deleteNote(id)` → Remove note from store
- `getNotes(options?)` → Return non-archived notes (or all with flag)
- `searchNotes(query)` → Case-insensitive search
- `promoteTranscriptionToNote(entry)` → Convert history → note
- `createNoteFromClipboard()` → Read clipboard → create note

7. IPC Handlers

Main process ↔ Renderer communication:

Channel	Type	Purpose
<code>get-notes</code>	handle	Fetch all non-archived notes
<code>search-notes</code>	handle	Search by query string
<code>create-note</code>	handle	Create text note from UI input
<code>update-note</code>	handle	Update note fields
<code>delete-note</code>	handle	Delete a note
<code>promote-to-note</code>	handle	History entry → note
<code>notes-updated</code>	event	Notify UI to refresh

Table 2: IPC communication channels

Phase 1 Deliverables

1. Working Command Mode with Right Option hotkey
2. Voice commands: save notes, save clipboard, open notes
3. Complete Notes Page UI with search and management
4. Save-as-note from dictation history
5. Local storage with 500-note capacity
6. Zero external API dependencies (all local)

Why Start Here

- Establishes foundational architecture
 - Delivers immediate user value (basic note capture)
 - No AI/cloud dependencies → fast, private, reliable
 - Users can start building note collection for Phase 2+ features
-

Phase 2 — AudioPen-Level Intelligence (AI Cleanup + Styles + Auto-tags)

Goal

Transform raw voice notes into immediately useful, well-formatted outputs — moving beyond simple transcription.

Core Features

1. AI Clean-up (Voice Note Enhancement)

Flow:

1. User creates voice note via Command Mode
2. Note saved with raw ASR transcript
3. Background process queues LLM cleanup job
4. Groq Llama processes note:
 - Remove filler words ("umm", "uh", "like")
 - Fix grammar and punctuation
 - Reorder into logical paragraphs
 - Improve sentence structure
5. Clean version stored alongside raw version
6. UI shows toggle: Raw vs Clean view

Implementation: Async background job, non-blocking, user sees raw content immediately.

2. Style Presets (Output Reformatting)

Available styles:

Style Preset	Output Format
Default	Coherent paragraphs, clean writing
Bullet List	Structured bullet points
Action Items	Extracted tasks only ("Do X", "Call Y")
Casual Memo	Friendly informal email style
Formal Doc	Technical documentation format
Tweet Thread	Twitter-style numbered tweets

Table 3: Phase 2 style presets

Voice commands for styling:

- "Turn last note into action items"
- "Restyle as email"
- "Convert to bullet points"
- "Make it formal"

UI interaction: One-click style selector in note detail view; instant re-rendering.

3. Super Summary (Multi-Note Synthesis)

Feature flow:

1. User selects multiple notes (checkboxes in notes list)
2. Click "Super Summary" button
3. AI reads all selected notes
4. Generates comprehensive combined summary
5. Shows common themes, key decisions, action items

Use case: Consolidate 10+ scattered thoughts about one project into single coherent overview.

4. AI Task Extraction

Automatic detection:

- AI scans note content for task-like patterns
- Extracts items with person-action-deadline structure
- Displays in "Tasks" section within note
- Format: "Person — Action — When"

Example extraction:

Note content: *"Need to call Rahul tomorrow about the deck, and update pricing slides by Friday"*

Extracted tasks:

- Call Rahul — tomorrow — about deck
- Update pricing slides — Friday

5. Basic Auto-tagging

AI-powered tag suggestion:

1. Note saved
2. Background AI analysis extracts 3–5 topic tags
3. Tags shown as chips in UI
4. User can approve, edit, or remove suggested tags
5. Accepted tags stored with note for future search/filtering

Tag examples: "StayFree", "YC Application", "Physics Exam", "Client Meeting", "Feature Ideas"

6. Hybrid Search (Keyword + Semantic)

Enhanced search capabilities:

- Maintains local vector index of note embeddings
- Single search box supports both:
 - Exact keyword matching (existing)
 - Semantic similarity search (new)
- Results ranked by combined score
- Works even when exact phrase not present

Example: Search "YC demo plan" finds notes about "Y Combinator presentation strategy" even without exact match.

Implementation: Small local SQLite database with vector embeddings; no cloud dependency.

Phase 2 Deliverables

1. AI cleanup for all voice notes (raw + clean versions)
2. 6 style presets with one-click restyling
3. Super Summary for multi-note consolidation
4. Automatic task extraction with dedicated UI section
5. AI-suggested tags with manual approval
6. Semantic search alongside keyword search
7. Voice commands for style operations

Technical Requirements

- Groq Llama API integration (already used for dictation formatting)
 - Local embedding model for semantic search
 - SQLite database for vector storage
 - Background job queue for async AI processing
-

Phase 3 — [Mem.ai](#) + Voicenotes-Level Brain (Ask AI + Smart Organization)

Goal

Transform StayFree into an AI personal assistant — notes become queryable knowledge base with intelligent recall and proactive suggestions.

Core Features

1. Notes Chat / "Ask StayFree"

Conversational note retrieval:

1. New "Chat" tab in Notes page
2. User types or speaks natural language queries
3. AI retrieves relevant notes using RAG (Retrieval-Augmented Generation)
4. Synthesizes answer from multiple notes
5. Cites specific notes in response

Example queries:

- "What did I decide about command mode hotkeys last week?"
- "Summarize all notes about physics exam prep from last month"
- "Draft a YC update based on my last 3 investor call notes"
- "What were my action items from yesterday's meeting?"

Implementation: Vector similarity search → retrieve top-K notes → LLM synthesis with context.

2. Voice-to-Voice Recall

Hands-free query flow:

- 1. Hold Right Option (Command Mode)
- 2. Ask question: "What were my StayFree feature ideas?"
- 3. AI transcribes question → searches notes → generates answer
- 4. Speaks answer back (text-to-speech)
- 5. Shows relevant notes in overlay

Use case: Driving, cooking, coding — when keyboard not accessible.

3. Advanced Command Mode Queries

Extended voice commands:

Voice Command	Action
"Find notes about [topic]"	Open filtered notes list
"Summarize my notes about [topic]"	Generate summary overlay
"Show my action items"	Display all extracted tasks
"What did I capture yesterday?"	Time-filtered search

Table 4: Phase 3 advanced commands

Natural language parsing: Regex patterns extended with optional LLM fallback for complex queries.

4. Related Notes

Automatic note linking:

- Note detail view shows "Related" section
- AI finds semantically similar notes
- Displays 3–5 most relevant connections
- Click to open related note

Algorithm: Cosine similarity on note embeddings; threshold-based filtering.

5. Collections / Smart Topics

Auto-grouping without folders:

1. AI analyzes all notes periodically
2. Detects common themes and topics
3. Suggests Collections: "Physics Exam", "StayFree Dev", "YC Prep"
4. One note can belong to multiple collections
5. User can edit, merge, or dismiss suggested collections

UI: Collection tags, filterable views, dedicated collection pages.

Inspired by: [Mem.ai](#)'s Collections — fluid, non-hierarchical organization.

6. Proactive Resurfacing (Heads Up)

Context-aware note suggestions:

- Monitors active application (e.g., VS Code, Safari, Notes app)
- Detects current context (coding, browsing YC site, writing)
- Surfaces relevant past notes in sidebar or tray widget
- Non-intrusive: small badge notification

Examples:

- Coding in VS Code → suggests ASR pipeline notes
- Safari on YC website → suggests YC application notes
- Apple Notes app open → suggests recent dictation notes

Implementation: Active window monitoring + content analysis + embedding similarity.

7. Custom Writing Style Training

Personalized AI output:

1. User pastes examples of their own writing (emails, blog posts)
2. AI learns personal writing style patterns
3. "My Style" preset added to style selector
4. Future note cleanups and rewrites match user's voice

Benefit: AI outputs don't sound generic — they sound like the user wrote them.

Phase 3 Deliverables

- 1. Full Notes Chat interface with RAG-powered Q&A
- 2. Voice-to-voice recall via Command Mode
- 3. Advanced natural language commands
- 4. Related notes auto-linking
- 5. Smart Collections with auto-grouping
- 6. Proactive note resurfacing based on context
- 7. Custom writing style training and "My Style" preset

Technical Requirements

- Enhanced vector database with larger capacity
- LLM with longer context window for chat
- Active window monitoring API (macOS accessibility)
- Text-to-speech integration for voice responses
- Style fine-tuning or few-shot prompting for custom writing

Phase 4 — Automation + Integrations + Sync

Goal

Add workflow glue and ecosystem connectivity — power users and teams can integrate StayFree into existing productivity stacks.

Core Features

1. Task Automation Integrations

External app connections:

Integration	Functionality
Apple Reminders	Push tasks with due dates
Todoist	Create tasks from notes
Notion	Sync notes to database
Things 3	Add tasks to projects

Table 5: Task management integrations

Voice commands:

- "Create tasks from last meeting note"
- "Remind me about this next Monday"
- "Send these action items to Todoist"

2. WhatsApp Bot Integration

Remote note capture:

1. User connects WhatsApp account to StayFree bot
2. Send voice message to bot on WhatsApp
3. Bot transcribes → creates note in StayFree
4. Note synced to desktop app
5. User receives confirmation message

Use case: Capture ideas on-the-go without opening laptop — WhatsApp message becomes StayFree note.

Inspired by: Voicenotes WhatsApp bot feature.

3. Cross-App Export

Export destinations:

- Obsidian vault (Markdown files)
- Notion page creation
- PDF export (formatted document)
- Gmail draft from note content
- Google Docs creation

Voice commands:

- "Send this note to Notion"
- "Export my exam notes as PDF"
- "Create Gmail draft from last note"

4. Webhooks / Zapier Support

Automation workflows:

1. User configures webhook URL in settings
2. Every new note triggers webhook POST
3. Zapier receives note data
4. Custom automation: send to Slack, add to Airtable, email summary, etc.

Use case: Every client meeting note auto-posts to project Slack channel.

Inspired by: AudioPen webhooks + Voicenotes Zapier integration.

5. Optional Encrypted Cloud Sync

Cross-device synchronization:

- Local-first architecture preserved
- Opt-in encrypted sync to cloud
- Mac + Windows sync seamlessly
- End-to-end encryption (user controls keys)
- Offline-first: works without internet
- Conflict resolution for simultaneous edits

Privacy guarantee: Notes encrypted before leaving device; server cannot read content.

Implementation options:

- Custom backend (AWS S3 + Lambda)
- Firebase with client-side encryption
- Supabase with RLS policies

Phase 4 Deliverables

1. Apple Reminders + Todoist + Notion task integrations
2. WhatsApp bot for remote note capture
3. Export to Obsidian, Notion, PDF, Gmail, Google Docs
4. Webhook support for custom automation
5. Zapier-compatible API
6. Optional encrypted cloud sync (Mac ↔ Windows)

Business Model Implications

Phase 4 features enable **monetization tiers**:

- Free tier: Phases 1–3 features, local-only
 - Pro tier: Cloud sync, advanced integrations, unlimited notes
 - Team tier: Shared collections, collaboration features
-

Implementation Strategy

Sequential vs Parallel Development

Recommended approach: Sequential phases with iterative user feedback.

1. Build Phase 1 → ship to beta users → gather feedback
2. Incorporate feedback → build Phase 2 → ship → validate
3. Repeat for Phases 3 and 4

Why sequential:

- Each phase builds on previous foundation
- Real user feedback shapes later features
- Faster time-to-market for core value
- Reduces wasted development on unused features

Technology Stack

Current stack (dictation app):

- Electron (cross-platform desktop)
- TypeScript (type safety)
- React (renderer UI)
- electron-store (local persistence)
- uiohook-napi (global hotkeys)
- Groq Whisper (ASR)
- Groq Llama (formatting)

Additions for Phases 2–4:

- SQLite (vector embeddings storage)

- Sentence-transformers (local embeddings)
- Node.js background workers (async AI jobs)
- Zapier webhook handlers
- WhatsApp Business API (Phase 4)
- Cloud storage SDK (Phase 4 sync)

Performance Targets

Operation	Target Latency
Voice command recognition	< 1 second
Note creation (voice)	< 2 seconds
AI cleanup generation	< 5 seconds (async)
Search results display	< 200ms
Notes Chat response	< 3 seconds
Super Summary generation	< 10 seconds

Table 6: Performance benchmarks

Feature Comparison Matrix

StayFree vs Competition (Post Phase 4)

Feature	Voicenotes	Mem.ai	AudioPen	StayFree
OS-level dictation	No	No	No	Yes
Voice note capture	Yes	Yes	Yes	Yes
AI cleanup/rewrite	No	No	Yes	Yes
Style presets	No	No	Yes	Yes
Ask AI / Chat	Yes	Yes	No	Yes
Auto-organization	No	Yes	No	Yes
Proactive resurface	No	Yes	No	Yes
Custom writing style	No	No	Yes	Yes
WhatsApp integration	Yes	No	No	Yes
Local-first/private	No	No	No	Yes
Command Mode (OS-level)	No	No	No	Yes
Cross-platform (Mac+Win)	Yes	Yes	Yes	Yes

Table 7: Competitive feature matrix

Unique Differentiators

StayFree's competitive advantages post-implementation:

- 1. Only app combining Wispr-level dictation + Mem-level brain**
 - Wispr: great dictation, weak notes
 - Mem/Voicenotes: great notes, no OS-level input
 - StayFree: best of both
- 2. Privacy-first local intelligence**
 - All competitors cloud-dependent
 - StayFree works fully offline (Phases 1–3)
 - Cloud sync is optional, encrypted
- 3. Command Mode as OS-level AI assistant**
 - Voice commands work system-wide
 - Not just note-taking, but full personal assistant foundation
 - Foundation for future phases: calendar management, email drafting, web research
- 4. Developer-friendly**
 - Code dictation already strong (existing feature)

- Technical documentation style preset
 - Hinglish + code-switching support (future)
-

Next Steps

Phase 1 Implementation Checklist

Before starting development:

- ☐ Review existing codebase structure
- ☐ Set up development branch for notes feature
- ☐ Create Notes data model schema
- ☐ Design Notes Page UI mockups
- ☐ Define IPC contract (main ↔ renderer)

Development sequence:

1. Backend: Notes CRUD service + electron-store integration
2. Hotkeys: Right Option detection + Command Mode pipeline
3. Parser: Regex command recognition
4. IPC: Handler registration + preload API
5. UI: Notes Page components (input, cards, detail modal, search)
6. Integration: Save-as-note from history
7. Widget: Visual mode distinction
8. Testing: End-to-end command flow validation

Estimated timeline: 2–3 weeks for Phase 1 (assuming part-time development)

Success Metrics

Phase 1 validation criteria:

- User can create voice note via Command Mode in < 2 seconds
- User can search and find specific note in < 5 seconds
- Zero crashes during 100 note creation cycle
- Notes persist across app restarts
- UI responsive with 500 notes loaded

Phase 2+ metrics will be defined based on Phase 1 user feedback and usage patterns.

Conclusion

This four-phase roadmap transforms StayFree from a dictation tool into a comprehensive AI personal assistant with best-in-class note intelligence. Each phase delivers incremental value while building toward the complete vision.

Key principles maintained throughout:

- Speed: < 1 second for core operations
- Privacy: Local-first, cloud-optional
- Quality: AudioPen-level output polish
- Intelligence: Mem/Voicenotes-level AI features
- Integration: Wispr-level OS connectivity

The foundation is already strong (dictation pipeline proven), and the architecture supports natural extension into notes and beyond.

Implementation can begin immediately with Phase 1, with subsequent phases informed by real user behavior and feedback.

StayFree — Smart Notes Feature Implementation Roadmap

Document Overview

This document outlines the complete phased implementation plan for adding Smart Notes and Command Mode features to StayFree, transforming it from a dictation-only app into an AI-powered personal assistant with intelligent note-taking capabilities.

Current State: StayFree is a macOS voice dictation tray app (hold Left Option → record → transcribe → format → auto-paste)

Target State: Wispr Flow-level dictation + [Mem.ai/Voicenotes-level](#) smart notes in a single tray app

Total Implementation Phases: 4

Phase 1 — Smart Notes Foundation (Regex-based, Minimal AI)

Goal

Turn StayFree from "dictation-only" into "dictation + solid notes hub" with zero UI friction.

Core Components

1. Data Model

- **Notes storage** in electron-store
- **Note interface** structure:
 - id: crypto.randomUUID()
 - title: Auto-generated from first ~60 chars, user-editable
 - content: Formatted note text
 - rawContent: Original raw transcription
 - createdAt: Date.now()
 - updatedAt: Date.now()
 - source: "voice" | "text" | "clipboard" | "transcription"
 - pinned: boolean
 - archived: boolean
 - tags: string[] (empty initially, used in Phase 2)
- **Storage limit**: 500 notes maximum

2. Command Mode Hotkey Setup

- **Right Option key** detection (separate from Left Option dictation)
- **Visual distinction** in widget:
 - Dictation mode (Left Option): Red color
 - Command mode (Right Option): Purple/blue color
- **Recording pipeline**: Same audio capture, different post-processing path

3. Voice Commands (Regex-based Parsing)

Command patterns:

Voice Command	Action
"save to notes: [content]"	Create note with spoken content
"note: [content]"	Create note with spoken content
"save clipboard to notes"	Read clipboard → create note
"open notes" / "show notes"	Open Settings window on Notes tab

Table 8: Phase 1 Command Mode voice commands

Implementation approach: Fast regex pattern matching (0ms vs 200-400ms LLM parsing), predictable behavior.

4. Notes Page UI

Layout structure:

- Top section: Text input field + "+" button for keyboard note creation
- Pinned section: Horizontally scrollable cards for pinned notes
- Recents section: Main notes list with search bar + view toggle
- Empty state: Helpful message teaching Command Mode usage

Note Card components:

- Title (auto-generated, editable)
- 2-line content preview
- Timestamp
- Source icon (microphone/keyboard/clipboard/history)
- Hover actions: pin, archive, delete, copy

Note Detail Modal:

- Editable title (click to edit)
- Editable content (textarea)
- Tags display (manual in Phase 1)
- Metadata: created date, source type
- Actions: copy, delete, archive

Search functionality:

- Debounced search bar (300ms delay)
- Case-insensitive keyword matching

- Searches across: title, content, tags

View options:

- Grid view (cards)
- List view (compact rows)

5. Transcription History Integration

"Save as Note" feature:

- Bookmark icon button in transcription history rows
- Click → converts history entry to note
- Source tagged as "transcription"
- Preserves original raw content

6. Notes CRUD Service

Core functions (src/main/notes.ts):

- `createNote(params)` → Generate UUID, auto-title, prepend to array, cap at 500
- `updateNote(id, updates)` → Partial update (title, content, pinned, archived, tags)
- `deleteNote(id)` → Remove note from store
- `getNotes(options?)` → Return non-archived notes (or all with flag)
- `searchNotes(query)` → Case-insensitive search
- `promoteTranscriptionToNote(entry)` → Convert history → note
- `createNoteFromClipboard()` → Read clipboard → create note

7. IPC Handlers

Main process ↔ Renderer communication:

Channel	Type	Purpose
get-notes	handle	Fetch all non-archived notes
search-notes	handle	Search by query string
create-note	handle	Create text note from UI input
update-note	handle	Update note fields
delete-note	handle	Delete a note
promote-to-note	handle	History entry → note
notes-updated	event	Notify UI to refresh

Table 9: IPC communication channels

Phase 1 Deliverables

1. Working Command Mode with Right Option hotkey
2. Voice commands: save notes, save clipboard, open notes
3. Complete Notes Page UI with search and management
4. Save-as-note from dictation history
5. Local storage with 500-note capacity
6. Zero external API dependencies (all local)

Why Start Here

- Establishes foundational architecture
- Delivers immediate user value (basic note capture)
- No AI/cloud dependencies → fast, private, reliable
- Users can start building note collection for Phase 2+ features

Phase 2 — AudioPen-Level Intelligence (AI Cleanup + Styles + Auto-tags)

Goal

Transform raw voice notes into immediately useful, well-formatted outputs — moving beyond simple transcription.

Core Features

1. AI Clean-up (Voice Note Enhancement)

Flow:

1. User creates voice note via Command Mode
2. Note saved with raw ASR transcript
3. Background process queues LLM cleanup job
4. Groq Llama processes note:
 - Remove filler words ("umm", "uh", "like")
 - Fix grammar and punctuation
 - Reorder into logical paragraphs
 - Improve sentence structure
5. Clean version stored alongside raw version
6. UI shows toggle: Raw vs Clean view

Implementation: Async background job, non-blocking, user sees raw content immediately.

2. Style Presets (Output Reformatting)

Available styles:

Style Preset	Output Format
Default	Coherent paragraphs, clean writing
Bullet List	Structured bullet points
Action Items	Extracted tasks only ("Do X", "Call Y")
Casual Memo	Friendly informal email style
Formal Doc	Technical documentation format
Tweet Thread	Twitter-style numbered tweets

Table 10: Phase 2 style presets

Voice commands for styling:

- "Turn last note into action items"
- "Restyle as email"
- "Convert to bullet points"
- "Make it formal"

UI interaction: One-click style selector in note detail view; instant re-rendering.

3. Super Summary (Multi-Note Synthesis)

Feature flow:

1. User selects multiple notes (checkboxes in notes list)
2. Click "Super Summary" button
3. AI reads all selected notes
4. Generates comprehensive combined summary
5. Shows common themes, key decisions, action items

Use case: Consolidate 10+ scattered thoughts about one project into single coherent overview.

4. AI Task Extraction

Automatic detection:

- AI scans note content for task-like patterns
- Extracts items with person-action-deadline structure
- Displays in "Tasks" section within note
- Format: "Person — Action — When"

Example extraction:

Note content: *"Need to call Rahul tomorrow about the deck, and update pricing slides by Friday"*

Extracted tasks:

- Call Rahul — tomorrow — about deck
- Update pricing slides — Friday

5. Basic Auto-tagging

AI-powered tag suggestion:

1. Note saved
2. Background AI analysis extracts 3–5 topic tags
3. Tags shown as chips in UI
4. User can approve, edit, or remove suggested tags

5. Accepted tags stored with note for future search/filtering

Tag examples: "StayFree", "YC Application", "Physics Exam", "Client Meeting", "Feature Ideas"

6. Hybrid Search (Keyword + Semantic)

Enhanced search capabilities:

- Maintains local vector index of note embeddings
- Single search box supports both:
 - Exact keyword matching (existing)
 - Semantic similarity search (new)
- Results ranked by combined score
- Works even when exact phrase not present

Example: Search "YC demo plan" finds notes about "Y Combinator presentation strategy" even without exact match.

Implementation: Small local SQLite database with vector embeddings; no cloud dependency.

Phase 2 Deliverables

1. AI cleanup for all voice notes (raw + clean versions)
2. 6 style presets with one-click restyling
3. Super Summary for multi-note consolidation
4. Automatic task extraction with dedicated UI section
5. AI-suggested tags with manual approval
6. Semantic search alongside keyword search
7. Voice commands for style operations

Technical Requirements

- Groq Llama API integration (already used for dictation formatting)
 - Local embedding model for semantic search
 - SQLite database for vector storage
 - Background job queue for async AI processing
-

Phase 3 — [Mem.ai](#) + Voicenotes-Level Brain (Ask AI + Smart Organization)

Goal

Transform StayFree into an AI personal assistant — notes become queryable knowledge base with intelligent recall and proactive suggestions.

Core Features

1. Notes Chat / "Ask StayFree"

Conversational note retrieval:

1. New "Chat" tab in Notes page
2. User types or speaks natural language queries
3. AI retrieves relevant notes using RAG (Retrieval-Augmented Generation)
4. Synthesizes answer from multiple notes
5. Cites specific notes in response

Example queries:

- "What did I decide about command mode hotkeys last week?"
- "Summarize all notes about physics exam prep from last month"
- "Draft a YC update based on my last 3 investor call notes"
- "What were my action items from yesterday's meeting?"

Implementation: Vector similarity search → retrieve top-K notes → LLM synthesis with context.

2. Voice-to-Voice Recall

Hands-free query flow:

1. Hold Right Option (Command Mode)
2. Ask question: "What were my StayFree feature ideas?"
3. AI transcribes question → searches notes → generates answer
4. Speaks answer back (text-to-speech)
5. Shows relevant notes in overlay

Use case: Driving, cooking, coding — when keyboard not accessible.

3. Advanced Command Mode Queries

Extended voice commands:

Voice Command	Action
"Find notes about [topic]"	Open filtered notes list
"Summarize my notes about [topic]"	Generate summary overlay
"Show my action items"	Display all extracted tasks
"What did I capture yesterday?"	Time-filtered search

Table 11: Phase 3 advanced commands

Natural language parsing: Regex patterns extended with optional LLM fallback for complex queries.

4. Related Notes

Automatic note linking:

- Note detail view shows "Related" section
- AI finds semantically similar notes
- Displays 3–5 most relevant connections
- Click to open related note

Algorithm: Cosine similarity on note embeddings; threshold-based filtering.

5. Collections / Smart Topics

Auto-grouping without folders:

1. AI analyzes all notes periodically
2. Detects common themes and topics
3. Suggests Collections: "Physics Exam", "StayFree Dev", "YC Prep"
4. One note can belong to multiple collections
5. User can edit, merge, or dismiss suggested collections

UI: Collection tags, filterable views, dedicated collection pages.

Inspired by: [Mem.ai](#)'s Collections — fluid, non-hierarchical organization.

6. Proactive Resurfacing (Heads Up)

Context-aware note suggestions:

- Monitors active application (e.g., VS Code, Safari, Notes app)
- Detects current context (coding, browsing YC site, writing)
- Surfaces relevant past notes in sidebar or tray widget
- Non-intrusive: small badge notification

Examples:

- Coding in VS Code → suggests ASR pipeline notes
- Safari on YC website → suggests YC application notes
- Apple Notes app open → suggests recent dictation notes

Implementation: Active window monitoring + content analysis + embedding similarity.

7. Custom Writing Style Training

Personalized AI output:

1. User pastes examples of their own writing (emails, blog posts)
2. AI learns personal writing style patterns
3. "My Style" preset added to style selector
4. Future note cleanups and rewrites match user's voice

Benefit: AI outputs don't sound generic — they sound like the user wrote them.

Phase 3 Deliverables

1. Full Notes Chat interface with RAG-powered Q&A
2. Voice-to-voice recall via Command Mode
3. Advanced natural language commands
4. Related notes auto-linking
5. Smart Collections with auto-grouping
6. Proactive note resurfacing based on context
7. Custom writing style training and "My Style" preset

Technical Requirements

- Enhanced vector database with larger capacity
- LLM with longer context window for chat
- Active window monitoring API (macOS accessibility)
- Text-to-speech integration for voice responses
- Style fine-tuning or few-shot prompting for custom writing

Phase 4 — Automation + Integrations + Sync

Goal

Add workflow glue and ecosystem connectivity — power users and teams can integrate StayFree into existing productivity stacks.

Core Features

1. Task Automation Integrations

External app connections:

Integration	Functionality
Apple Reminders	Push tasks with due dates
Todoist	Create tasks from notes
Notion	Sync notes to database
Things 3	Add tasks to projects

Table 12: Task management integrations

Voice commands:

- "Create tasks from last meeting note"
- "Remind me about this next Monday"
- "Send these action items to Todoist"

2. WhatsApp Bot Integration

Remote note capture:

1. User connects WhatsApp account to StayFree bot
2. Send voice message to bot on WhatsApp

3. Bot transcribes → creates note in StayFree
4. Note synced to desktop app
5. User receives confirmation message

Use case: Capture ideas on-the-go without opening laptop — WhatsApp message becomes StayFree note.

Inspired by: Voicenotes WhatsApp bot feature.

Status: Not yet implemented — planned for Phase 4.

3. Cross-App Export

Export destinations:

- Obsidian vault (Markdown files)
- Notion page creation
- PDF export (formatted document)
- Gmail draft from note content
- Google Docs creation

Voice commands:

- "Send this note to Notion"
- "Export my exam notes as PDF"
- "Create Gmail draft from last note"

4. Webhooks / Zapier Support

Automation workflows:

1. User configures webhook URL in settings
2. Every new note triggers webhook POST
3. Zapier receives note data
4. Custom automation: send to Slack, add to Airtable, email summary, etc.

Use case: Every client meeting note auto-posts to project Slack channel.

Inspired by: AudioPen webhooks + Voicenotes Zapier integration.

5. Optional Encrypted Cloud Sync

Cross-device synchronization:

- Local-first architecture preserved
- Opt-in encrypted sync to cloud
- Mac + Windows sync seamlessly
- End-to-end encryption (user controls keys)
- Offline-first: works without internet
- Conflict resolution for simultaneous edits

Privacy guarantee: Notes encrypted before leaving device; server cannot read content.

Implementation options:

- Custom backend (AWS S3 + Lambda)
- Firebase with client-side encryption
- Supabase with RLS policies

Phase 4 Deliverables

1. Apple Reminders + Todoist + Notion task integrations
2. WhatsApp bot for remote note capture
3. Export to Obsidian, Notion, PDF, Gmail, Google Docs
4. Webhook support for custom automation
5. Zapier-compatible API
6. Optional encrypted cloud sync (Mac ↔ Windows)

Business Model Implications

Phase 4 features enable **monetization tiers**:

- Free tier: Phases 1–3 features, local-only
 - Pro tier: Cloud sync, advanced integrations, unlimited notes
 - Team tier: Shared collections, collaboration features
-

Implementation Strategy

Sequential vs Parallel Development

Recommended approach: Sequential phases with iterative user feedback.

1. Build Phase 1 → ship to beta users → gather feedback
2. Incorporate feedback → build Phase 2 → ship → validate
3. Repeat for Phases 3 and 4

Why sequential:

- Each phase builds on previous foundation
- Real user feedback shapes later features
- Faster time-to-market for core value
- Reduces wasted development on unused features

Technology Stack

Current stack (dictation app):

- Electron (cross-platform desktop)
- TypeScript (type safety)
- React (renderer UI)
- electron-store (local persistence)
- uiohook-napi (global hotkeys)
- Groq Whisper (ASR)
- Groq Llama (formatting)

Additions for Phases 2–4:

- SQLite (vector embeddings storage)
- Sentence-transformers (local embeddings)
- Node.js background workers (async AI jobs)
- Zapier webhook handlers
- WhatsApp Business API (Phase 4)
- Cloud storage SDK (Phase 4 sync)

Performance Targets

Operation	Target Latency
Voice command recognition	< 1 second
Note creation (voice)	< 2 seconds
AI cleanup generation	< 5 seconds (async)
Search results display	< 200ms
Notes Chat response	< 3 seconds
Super Summary generation	< 10 seconds

Table 13: Performance benchmarks

Feature Comparison Matrix

StayFree vs Competition (Post Phase 4)

Feature	Voicenotes	Mem.ai	AudioPen	StayFree
OS-level dictation	No	No	No	Yes
Voice note capture	Yes	Yes	Yes	Yes
AI cleanup/rewrite	No	No	Yes	Yes
Style presets	No	No	Yes	Yes
Ask AI / Chat	Yes	Yes	No	Yes
Auto-organization	No	Yes	No	Yes
Proactive resurface	No	Yes	No	Yes
Custom writing style	No	No	Yes	Yes
WhatsApp integration	Yes	No	No	Yes
Local-first/private	No	No	No	Yes
Command Mode (OS-level)	No	No	No	Yes
Cross-platform (Mac+Win)	Yes	Yes	Yes	Yes

Table 14: Competitive feature matrix

Unique Differentiators

StayFree's competitive advantages post-implementation:

1. **Only app combining Wispr-level dictation + Mem-level brain**
 - Wispr: great dictation, weak notes
 - Mem/Voicenotes: great notes, no OS-level input
 - StayFree: best of both
 2. **Privacy-first local intelligence**
 - All competitors cloud-dependent
 - StayFree works fully offline (Phases 1–3)
 - Cloud sync is optional, encrypted
 3. **Command Mode as OS-level AI assistant**
 - Voice commands work system-wide
 - Not just note-taking, but full personal assistant foundation
 - Foundation for future phases: calendar management, email drafting, web research
 4. **Developer-friendly**
 - Code dictation already strong (existing feature)
 - Technical documentation style preset
 - Hinglish + code-switching support (future)
-

Quick Phase Summary

All Features by Phase

Phase 1 — Foundation (Build NOW):

- Right Option hotkey for Command Mode
- Voice commands: save notes, save clipboard, open notes
- Notes Page UI: cards, search, pin/archive/delete
- Save-as-note from history
- Local storage (500 notes)

Phase 2 — Intelligence (After Phase 1 validated):

- AI cleanup (remove fillers, fix grammar)
- Style presets: bullets, action items, memo, formal, tweet
- Super Summary (multi-note synthesis)
- AI task extraction

- Auto-tagging (AI-suggested tags)
- Hybrid search (keyword + semantic)

Phase 3 — Brain (After Phase 2 validated):

- Notes Chat / Ask StayFree (RAG-powered Q&A)
- Voice-to-voice recall
- Advanced commands: find, summarize, show action items
- Related notes auto-linking
- Smart Collections (auto-grouping)
- Proactive resurfacing (context-aware suggestions)
- Custom writing style training

Phase 4 — Ecosystem (After Phase 3 validated):

- Task integrations: Apple Reminders, Todoist, Notion
 - WhatsApp bot (remote note capture)
 - Cross-app export: Obsidian, Notion, PDF, Gmail
 - Webhooks / Zapier support
 - Optional encrypted cloud sync (Mac ↔ Windows)
-

Next Steps

Phase 1 Implementation Checklist

Before starting development:

- ☐ Review existing codebase structure
- ☐ Set up development branch for notes feature
- ☐ Create Notes data model schema
- ☐ Design Notes Page UI mockups
- ☐ Define IPC contract (main ↔ renderer)

Development sequence:

1. Backend: Notes CRUD service + electron-store integration
2. Hotkeys: Right Option detection + Command Mode pipeline
3. Parser: Regex command recognition
4. IPC: Handler registration + preload API
5. UI: Notes Page components (input, cards, detail modal, search)

6. Integration: Save-as-note from history
7. Widget: Visual mode distinction
8. Testing: End-to-end command flow validation

Estimated timeline: 2–3 weeks for Phase 1 (assuming part-time development)

Success Metrics

Phase 1 validation criteria:

- User can create voice note via Command Mode in < 2 seconds
- User can search and find specific note in < 5 seconds
- Zero crashes during 100 note creation cycle
- Notes persist across app restarts
- UI responsive with 500 notes loaded

Phase 2+ metrics will be defined based on Phase 1 user feedback and usage patterns.

Conclusion

This four-phase roadmap transforms StayFree from a dictation tool into a comprehensive AI personal assistant with best-in-class note intelligence. Each phase delivers incremental value while building toward the complete vision.

Key principles maintained throughout:

- Speed: < 1 second for core operations
- Privacy: Local-first, cloud-optional
- Quality: AudioPen-level output polish
- Intelligence: Mem/Voicenotes-level AI features
- Integration: Wispr-level OS connectivity

The foundation is already strong (dictation pipeline proven), and the architecture supports natural extension into notes and beyond. Implementation can begin immediately with Phase 1, with subsequent phases informed by real user behavior and feedback.

